

Министерство науки и высшего образования РФ  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

## ОТЧЁТ

По лабораторной работе №2

По дисциплине «Логика и основы алгоритмизации в инженерных  
задачах»

На тему «Оценка времени выполнения программ»

Выполнили студенты группы  
25BVB3:

Сергеева С.М.

Иванова С.А.

Приняли:

к.т.н., доцент Юрова О. В.

к.т.н., доцент Деев М. В.

Пенза 2026

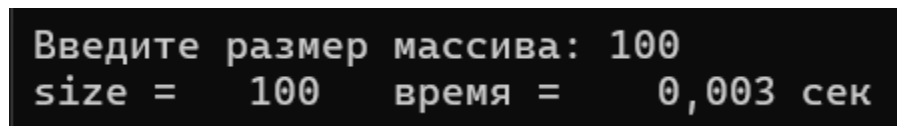
Цель: Оценить время выполнения программ с использованием библиотеки time.h.

Ход работы.

Задание 1: Вычислили порядок сложности программы, оценили время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки time.h для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000, построили график зависимости времени выполнения программы от размера матриц и сравнили полученный результат с теоретической оценкой.

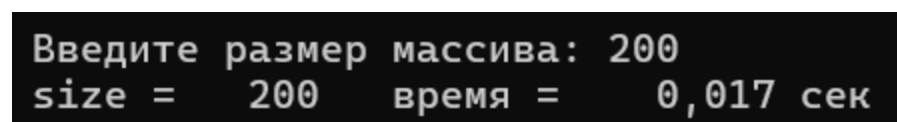
Описание программы:

Программа вычисляет произведение двух квадратных матриц размером  $\text{size} \times \text{size}$  и записывает результат в новую матрицу. Сначала пользователь вводит размер матрицы, затем генератор случайных чисел инициализируется через `srand(time(NULL))`, после чего обе исходные матрицы `a` и `b` заполняются случайными числами в диапазоне от 1 до 100 с помощью `rand() % 100 + 1`. Перед началом вычислений запускается таймер `start = clock()`, а после завершения всех циклов — останавливается `end = clock()`, и итоговое время выводится на экран как  $(\text{end} - \text{start}) / \text{CLOCKS\_PER\_SEC}$ .



```
Введите размер массива: 100
size = 100    время = 0,003 сек
```

Рисунок 1 – Результат умножения матриц с размером 100



```
Введите размер массива: 200
size = 200    время = 0,017 сек
```

Рисунок 2 – Результат умножения матриц с размером 200

```
Введите размер массива: 400
size = 400    время = 0,130 сек
```

Рисунок 3 – Результат умножения матриц с размером 400

```
Введите размер массива: 1000
size = 1000    время = 1,867 сек
```

Рисунок 4 – Результат умножения матриц с размером 1000

```
Введите размер массива: 2000
size = 2000    время = 15,542 сек
```

Рисунок 5 – Результат умножения матриц с размером 2000

```
Введите размер массива: 4000
size = 4000    время = 382,013 сек
```

Рисунок 6 – Результат умножения матриц с размером 4000

```
Введите размер массива: 10000
size = 10000    время = 10259,277 сек
```

Рисунок 7 – Результат умножения матриц с размером 10000

| Значения/Размер массивов | 100   | 200   | 400   | 1000   | 2000    | 4000    | 10000     |
|--------------------------|-------|-------|-------|--------|---------|---------|-----------|
| Теоретические            | 0,015 | 0,120 | 0,960 | 15,000 | 120,000 | 960,000 | 15000,000 |
| Практические             | 0,003 | 0,017 | 0,130 | 1,867  | 15,542  | 382,013 | 10259,277 |

Таблица 1 - Теоретические и практические значения времени

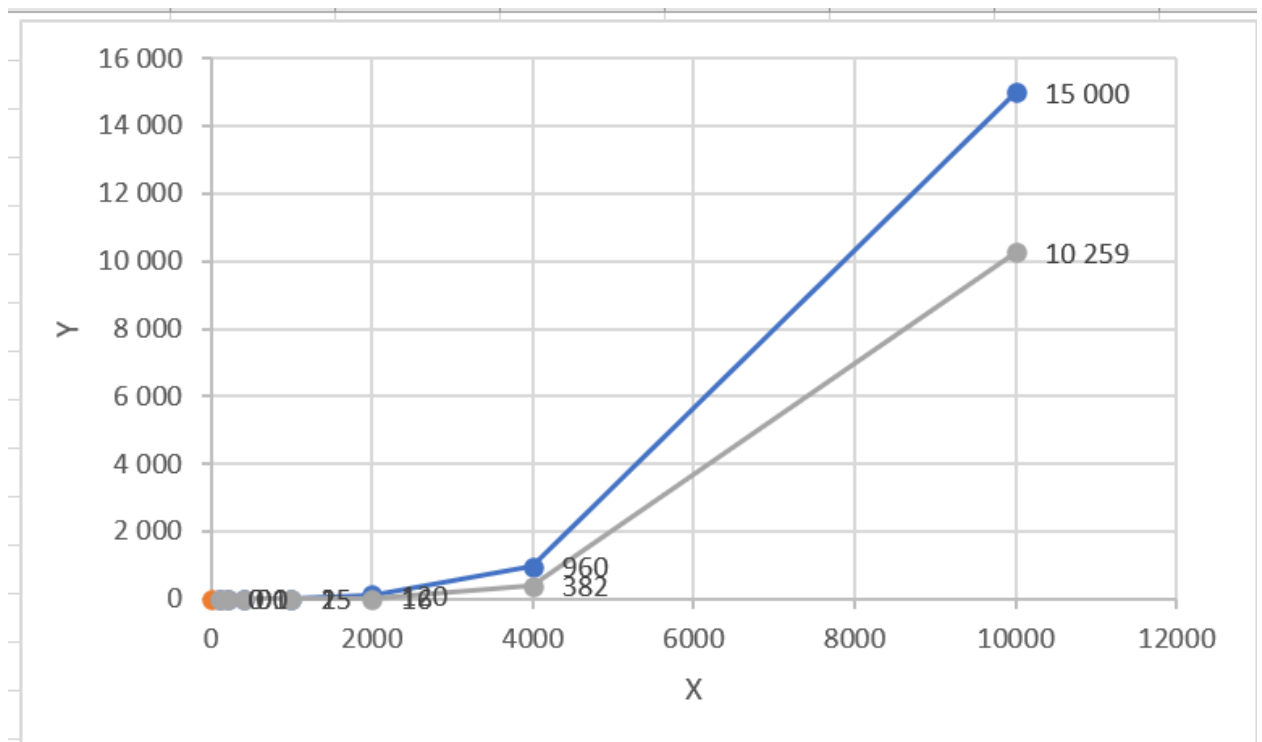


Рисунок 8 – График скорости выполнения

Каждый элемент  $s[i][j]$  вычисляется за  $n$  умножений, а всего элементов в матрице  $S$  -  $n^2$ , поэтому общее число операций умножения равно  $n^3$ , получаем кубическую сложность  $T(n) = O(n^3)$ .

Задание 2: Оценили время работы каждого из реализованных алгоритмов на случайном наборе значений массива, время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел, время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел, время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую, время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Описание программы:

Массив заполняется случайными числами от 1 до 100  
`items[i]=rand%100+1`. Генератор инициализируется через `srand(time(NULL))`.

Введите размер массива: 100

| Тип массива           | shell        | qs           | qsort        |
|-----------------------|--------------|--------------|--------------|
| Случайный             | 0,0000000000 | 0,0000000000 | 0,0000000000 |
| Возрастающий          | 0,0000000000 | 0,0000000000 | 0,0000000000 |
| Убывающий             | 0,0000000000 | 0,0000000000 | 0,0000000000 |
| Возрастающе-убывающий | 0,0000000000 | 0,0000000000 | 0,0000000000 |

Рисунок 9 – Вывод для массива размером 100

Введите размер массива: 1000

| Тип массива           | shell        | qs           | qsort        |
|-----------------------|--------------|--------------|--------------|
| Случайный             | 0,0000000000 | 0,0000000000 | 0,0000000000 |
| Возрастающий          | 0,0000000000 | 0,0000000000 | 0,0000000000 |
| Убывающий             | 0,0000000000 | 0,0000000000 | 0,0000000000 |
| Возрастающе-убывающий | 0,0000000000 | 0,0000000000 | 0,0000000000 |

Рисунок 10 – Вывод для массива размером 1000

Введите размер массива: 10000

| Тип массива           | shell        | qs           | qsort        |
|-----------------------|--------------|--------------|--------------|
| Случайный             | 0,0110000000 | 0,0010000000 | 0,0010000000 |
| Возрастающий          | 0,0000000000 | 0,0000000000 | 0,0020000000 |
| Убывающий             | 0,0210000000 | 0,0010000000 | 0,0010000000 |
| Возрастающе-убывающий | 0,0050000000 | 0,0000000000 | 0,0010000000 |

Рисунок 11 – Вывод для массива размером 10000

Введите размер массива: 100000

| Тип массива           | shell        | qs           | qsort        |
|-----------------------|--------------|--------------|--------------|
| Случайный             | 1,0230000000 | 0,0090000000 | 0,0120000000 |
| Возрастающий          | 0,0020000000 | 0,0040000000 | 0,0100000000 |
| Убывающий             | 2,0220000000 | 0,0030000000 | 0,0120000000 |
| Возрастающе-убывающий | 0,4830000000 | 0,0040000000 | 0,0110000000 |

Рисунок 12 – Вывод для массива размером 100000

Первая половина массива заполняется возрастающей последовательностью  $items[i]=i$ , вторая половина — убывающей  $items[i]=size-1-(i-size/2)$ .

После сортировок `shell`, `qs` массив восстанавливается через `memcpy` и сортируется стандартной функцией `qsort`. Время замеряется так же, как и для остальных алгоритмов. Это позволяет сравнить эффективность собственной реализации с оптимизированной библиотечной.

Вывод: В ходе лабораторной работы было выявлено следующее. Сортировка Шелла чувствительна к порядку элементов, на уже отсортированных она работает быстро (быстрее `Qs`), а на убывающих — медленно. Быстрая сортировка одинаково справляется как со случайными, так и с упорядоченными значениями, что подтверждает ее среднюю сложность  $O(n \log n)$ . Стандартная библиотечная функция уступает ручной по времени. Это связано с внутренним устройством библиотеки. В нашем случае до 10000 элементов выбор не так важен, но если значение больше лучше выбрать `Qs`.